

**LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

**MODUL 3
DESIGN PRINCIPLE (S.O.L.I.D)**

DISUSUN OLEH :

AMMAR BAGAS FATHURRAHMAN WANTORO 2350081008



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI	1
BAB I. HASIL PRAKTIKUM	2
I.1 Interface Segregation Interface	2
I.1.A. Source Code	2
I.1.B. Analisa	4
I.2 Depedency Inversion Principle	5
I.2.A. Source code	5
I.2.B. Analisa	7
BAB II. TUGAS PRAKTIKUM	8
II.1 Tugas	8
II.1.A. Soal	8
II.1.B. Jawab	8
BAB III. KESIMPULAN	10

BAB I. HASIL PRAKTIKUM

I.1 Interface Segregation Interface

I.1.A. Source Code

A. Vehicle Interface

```
interface VehicleInterface {  
  
    void drive();  
  
    void stop();  
  
    void refuel();  
  
    void openDoors();  
  
}
```

B. Class Motorcycle

```
class Motorcycle implements VehicleInterface {  
  
    // Can be implemented  
  
    @Override  
    public void drive() {  
  
    }  
  
    @Override  
    public void stop() {  
  
    }  
  
    @Override  
    public void refuel() {  
  
    }  
  
    // Can not be implemented  
  
    @Override  
    public void openDoors() {  
  
    }  
  
}
```

C. Class car

```
class Car implements VehicleInterface {  
    // Can be implemented  
    @Override  
    public void drive() {  
    }  
    @Override  
    public void stop() {  
    }  
    @Override  
    public void refuel() {  
    }  
    @Override  
    public void openDoors() {  
    }  
}
```

D. Vehicle Interface

```
interface VehicleInterface {  
    void drive();  
    void stop();  
    void refuel();  
}
```

E. Interface door

```
interface DoorInterface {  
    void openDoors();  
}
```

F. VehicleInterface

```
class Motorcycle : VehicleInterface {  
  
    override fun drive() {}  
  
    override fun stop() {}  
  
    override fun refuel() {}  
  
}
```

G. Car

```
class Car implements VehicleInterface, DoorInterface {  
  
    @Override  
    public void drive() {  
  
    }  
  
    @Override  
    public void stop() {  
  
    }  
  
    @Override  
    public void refuel() {  
  
    }  
  
    @Override  
    public void openDoors() {  
  
    }  
  
}
```

I.1.B. Analisa

Analisis kode ini menunjukkan penerapan Interface Segregation Principle (ISP) yang efektif dengan memecah fungsionalitas kendaraan menjadi interface yang lebih spesifik. Tanpa ISP, kita mungkin akan memiliki satu fat interface (misalnya VehicleInterface yang juga berisi openDoors()). Ini akan memaksa kelas Motorcycle untuk mengimplementasikan metode openDoors(), yang tidak relevan

baginya, sehingga menghasilkan implementasi kosong atau exception yang menandakan desain yang buruk.

Dengan menerapkan ISP, fungsionalitas dipisah menjadi VehicleInterface (untuk drive, stop, refuel) dan DoorInterface (untuk openDoors). Desain ini sangat menguntungkan: kelas Car dapat mengimplementasikan kedua interface karena relevan, sementara kelas Motorcycle cukup mengimplementasikan VehicleInterface saja. Hasilnya adalah desain yang decoupled dan bersih, di mana setiap kelas hanya bergantung pada metode yang benar-benar mereka gunakan, membuat sistem lebih mudah dipelihara dan dipahami.

I.2 Depedency Inversion Principle

I.2.A. Source code

A. Class Car

```
class Car {  
    private final Engine engine;  
  
    public Car(Engine engine) {  
        this.engine = engine;  
    }  
  
    void start() {  
        engine.start();  
    }  
}
```

B. Class Engine

```
class Engine {  
    void start() {  
    }  
}
```

C. Class Diesel Engine

```
class DieselEngine {  
    void start() {  
    }  
}
```

D. Petrol Engine

```
class PetrolEngine implements EngineInterface {  
    @Override  
    public void start() {  
    }  
}
```

E. Hybrid Engine

```
class HybridEngine implements EngineInterface {  
    @Override  
    public void start() {  
    }  
}
```

F. Diesel Engine

```
class DieselEngine implements EngineInterface {  
    @Override  
    public void start() {  
    }  
}
```

G. Class Main

```
class Main {  
  
    public static void main(String[] args) {  
  
        Car fuelCar = new Car(new PetrolEngine());  
  
        Car dieselCar = new Car(new DieselEngine());  
  
        Car hybridCar = new Car(new HybridEngine());  
  
        //Booom boom  
  
        fuelCar.start();  
  
        dieselCar.start();  
  
        hybridCar.start();  
  
    }  
  
}
```

I.2.B. Analisa

Analisis kode ini menunjukkan penerapan Dependency Inversion Principle (DIP) yang memisahkan modul high-level dari low-level. Dalam kasus ini, kelas Car adalah modul high-level (yang memiliki tujuan bisnis "berkendara"), sedangkan kelas-kelas seperti PetrolEngine dan DieselEngine adalah modul low-level (detail implementasi). Desain ini "membalik" ketergantungan: alih-alih Car bergantung langsung pada PetrolEngine, kini Car (modul high-level) dan semua jenis mesin (modul low-level) sama-sama bergantung pada satu abstraksi, yaitu EngineInterface.

Penerapan ini dicapai melalui Dependency Injection pada konstruktor Car, yang memaksa client (kelas Main) untuk "menyuntikkan" objek mesin yang spesifik. Manfaat utamanya adalah decoupling atau pelonggaran sistem secara drastis. Seperti yang terlihat di kelas Main, kelas Car sama sekali tidak perlu diubah-ubah hanya untuk berganti mesin. Kita bisa dengan bebas membuat mobil baru dengan HybridEngine atau DieselEngine selama mereka mematuhi kontrak EngineInterface, membuat kode ini sangat fleksibel, mudah di-maintenance, dan mudah untuk diuji secara terisolasi.

BAB II. TUGAS PRAKTIKUM

II.1 Tugas

II.1.A. Soal

1. Pada Latihan 4.2.1 ISP, Jelaskan alasan kenapa bentuk/design codenya harus diubah?
2. Ubah Class Motorcycle menggunakan fungsi java.
3. Pada Latihan 4.2.2 DPI, Jelaskan alasan kenapa bentuk/design codenya harus diubah?

II.1.B. Jawab

1. Desain kode tersebut (yang sudah menerapkan ISP) adalah solusi untuk menghindari masalah "fat interface" atau interface gemuk. Alasan utamanya adalah: jika kita tidak memisahkan interface, kita mungkin akan berakhir dengan satu VehicleInterface besar yang berisi drive(), stop(), refuel(), dan openDoors(). Masalahnya, kelas Motorcycle kemudian dipaksa untuk mengimplementasikan metode openDoors() yang sama sekali tidak relevan baginya, yang kemungkinan akan menghasilkan implementasi kosong atau exception. Ini melanggar ISP ("clients should not be forced to depend on methods they do not use"), sehingga desain harus diubah dengan memecah interface menjadi VehicleInterface dan DoorInterface, agar setiap kelas (Motorcycle atau Car) hanya bergantung pada metode yang benar-benar mereka butuhkan.

2. Code Class Motorcycle

```
package ISP;

public class Motorcycle implements VehicleInterface {

    @Override
    public void drive() {

    }

    @Override
    public void stop() {
```

```
    }

    @Override
    public void refuel() {

    }

}
```

3. Tanpa desain DIP, kelas Car (modul high-level) akan secara langsung membuat instansi dari mesin spesifik, misalnya `new PetrolEngine()` di dalam konstruktornya. Masalahnya, ini membuat Car "terikat mati" pada `PetrolEngine`. Jika kita ingin membuat Car dengan `DieselEngine`, kita dipaksa untuk mengubah kode di dalam kelas Car, yang melanggar Open/Closed Principle. Desain ini harus diubah ke DIP—di mana Car hanya bergantung pada abstraksi (`EngineInterface`)—agar Car menjadi fleksibel dan terdekopel (terpisah) dari detail implementasi mesin, sehingga kita bisa "menyuntikkan" mesin apa pun (`PetrolEngine`, `DieselEngine`) tanpa mengubah kelas Car sama sekali.

BAB III. KESIMPULAN

Praktikum ini telah mendemonstrasikan bagaimana Interface Segregation Principle (ISP) dan Dependency Inversion Principle (DIP) bekerja bersama untuk menciptakan arsitektur perangkat lunak yang sangat fleksibel dan decoupled.

Penerapan ISP mengajarkan pentingnya memecah fat interface (antarmuka gemuk) menjadi beberapa interface yang lebih kecil dan spesifik berdasarkan kebutuhan client. Seperti yang ditunjukkan pada kasus VehicleInterface dan DoorInterface, hal ini mencegah kelas seperti Motorcycle dipaksa untuk mengimplementasikan metode openDoors() yang tidak relevan, sehingga menghasilkan desain yang lebih bersih dan kohesif.

Sementara itu, DIP mengubah cara modul saling bergantung. Dengan "membalik" ketergantungan—di mana modul high-level (Car) dan low-level (PentrolEngine) kini sama-sama bergantung pada abstraksi (EngineInterface)—kami berhasil menghilangkan tight coupling. Penggunaan Dependency Injection memungkinkan sistem menjadi sangat modular, di mana komponen (seperti jenis mesin) dapat diganti-ganti dengan mudah tanpa mengubah kode di kelas Car.

Secara keseluruhan, kedua prinsip ini adalah kunci untuk membangun sistem yang tidak hanya mudah dipelihara dan diperluas, tetapi juga sangat mudah untuk diuji secara terisolasi.